

Danh sách tính năng hệ thống - Bản tổng hợp của nhóm

Đề tài: Nền tảng kiểm tra và cảnh báo rủi ro lừa đảo trực tuyến (Anti-Scam Platform)

Nhóm: Hùng, Khải, Kiên, Thắng

Tuần: 4 · Ngày tổng hợp: 10/08/2026

Mục đích của tài liệu này

Tuần 4 mỗi thành viên viết một file feature list riêng theo template chung. Khi ghép lại, nhóm phát hiện ba vấn đề: Kiên và Thắng khai trùng gần hết phần phân tích nội dung, ba bạn cùng khai bảng `scan_requests` và `risk_results` với enum khác nhau, và Slice 4 (báo cáo cộng đồng) chưa có người nhận dù ba slice còn lại đều cần dữ liệu từ nó.

File này gộp bốn bản riêng thành một danh sách duy nhất: giữ lại các tính năng nhóm thấy hợp lý và làm được trong phạm vi khóa luận, bỏ hoặc dời sang Phase 2 những phần vượt sức, và chốt luôn các chỗ bốn bản mô tả khác nhau về cùng một thứ. Từ tuần 5, đây là bản dùng để chia việc; bốn file cá nhân giữ lại làm hồ sơ quá trình.

Kiến trúc đã chốt ở tuần 3 và không thay đổi: Modular Monolith (Spring Boot) cho phần đồng bộ, worker qua RabbitMQ cho phần chậm, PostgreSQL làm nguồn sự thật, Redis cho cache và điều phối, MinIO lưu bằng chứng.

1. Phân công sau khi gộp

Slice	Nội dung	Người phụ trách
1	Auth, User, RBAC, Audit log	Hùng
2	Scan URL và lớp fetch an toàn (SSRF guard)	Khải
3a	Rule Engine và Text Analyzer	Kiên
3b	Entity Checker (phone, tài khoản) và QR Parser	Thắng
4	Community Report và Admin Dashboard (mức tối thiểu)	Hùng kiểm

Hai điều chỉnh so với bốn bản nộp riêng:

Tách Slice 3 làm hai. Bản của Kiên viết kỹ nhất phần `risk_rules`, `rule_conditions`, cơ chế nạp lại rule cache và trang admin quản lý rule; bản của Thắng viết kỹ nhất phần chuẩn hóa số điện thoại, số tài khoản, cách mask dữ liệu và component hiển thị kết quả dùng chung. Chia theo đúng thế mạnh đã thể hiện trong hai bản: Kiên lấy Rule Engine và Text, Thắng lấy Entity và QR. Ranh giới rõ ràng vì Text Analyzer gọi Entity Checker qua interface, không đọc trực tiếp bảng của nhau.

Slice 4 ghép vào Slice 1. Ba slice kia đều phụ thuộc bảng `scam_reports`, nếu để trống thì phone và bank checker không có dữ liệu để chấm điểm, coi như demo rỗng. Nhóm chỉ có 4 người nên không thể để một slice không chủ. Slice 1 của Hùng đã gần xong và bản thân Slice 4 dùng lại RBAC, `audit_logs` cùng trang admin mà Hùng đã dựng, nên chi phí thêm là thấp nhất. Bù lại, Slice 4 chỉ làm mức tối thiểu: nhận báo cáo, admin duyệt, chuyển thành risk entity, và một trang thống kê đơn giản. Các phần như tính điểm uy tín người báo cáo hay export báo cáo định kỳ dời sang Phase 2.

2. Chuẩn dùng chung

Phần này phải chốt trước khi bốn người viết code, vì cả bốn slice đều đọc và ghi vào cùng một bộ bảng.

2.1. Mô hình kết quả trả về

Mọi endpoint scan, dù là URL, text, phone, tài khoản hay QR, đều trả về cùng một cấu trúc. Frontend nhờ vậy chỉ cần một component hiển thị.

```
{
  "scanId": "uuid",
  "inputType": "URL | TEXT | PHONE | BANK_ACCOUNT | QR",
  "status": "PENDING | PROCESSING | COMPLETED | FAILED",
  "riskScore": 92,
  "riskLevel": "SAFE | CAUTION | DANGER",
  "confidence": 0.85,
  "summary": "Trang này có nhiều dấu hiệu giả mạo ngân hàng",
  "evidences": [
    {
      "ruleCode": "DOMAIN_TYPOSQUATTING",
      "ruleName": "Tên miền gần giống thương hiệu thật",
      "score": 30,
      "severity": "HIGH",
      "description": "Chỉ khác vietcombank.com.vn 2 ký tự",
      "matchedValueMasked": "v1etcombank***.tk",
      "source": "RULE_ENGINE"
    }
  ],
  "recommendation": {
    "action": "DO_NOT_PROCEED",
    "message": "Không nhập thông tin đăng nhập hay OTP vào trang này",
    "reasons": ["Tên miền mới đăng ký 5 ngày", "Form yêu cầu OTP"],
    "nextSteps": ["Gọi tổng đài ngân hàng theo số in trên thẻ để xác minh"]
  },
  "ruleSnapshotVersion": "v12"
}
```

Ngưỡng phân loại: 0 đến 29 là SAFE (xanh), 30 đến 59 là CAUTION (vàng), 60 trở lên là DANGER (đỏ). Điểm cộng dồn từ các rule khớp, sau đó chặn trong khoảng 0 đến 100.

Khi một scan gọi sang scan khác (tin nhắn có chứa link, QR chứa link hoặc số tài khoản), điểm của scan con được nhân hệ số 0.7 rồi cộng vào scan cha. Lý do nhân hệ số: một link rủi ro xuất hiện trong tin nhắn thì đáng ngờ, nhưng bản thân tin nhắn vẫn cần dấu hiệu riêng để bị chấm mức nguy hiểm cao nhất.

2.2. Các enum chốt

Bốn bản riêng mô tả khác nhau ở đây, nhóm lấy bản đầy đủ nhất trong mỗi trường hợp:

Trường	Giá trị choắt	Ghi chú
input_type	URL , TEXT , PHONE , BANK_ACCOUNT , QR	
scan_requests.status	PENDING , PROCESSING , COMPLETED , FAILED	Lấy bản của Thắng, có PENDING cho lúc đã tạo bản ghi nhưng chưa publish message
risk_level	SAFE , CAUTION , DANGER	
risk_entities.status	PENDING , VERIFIED , WHITELIST , RESOLVED	Lấy bản của Thắng vì phản ánh đúng quy trình kiểm duyệt, thay cho ACTIVE/INACTIVE
risk_entities.source	ADMIN_MANUAL , COMMUNITY_REPORT , FEED_IMPORT	Gộp hai bản, thêm cột feed_name để biết đến từ PhishTank hay OpenPhish
risk_rules.category	URL , TEXT , PHONE , BANK_ACCOUNT , QR , ENTITY , COMPOSITE	Hợp của hai bản
severity	LOW , MEDIUM , HIGH	

2.3. Lưu dữ liệu gốc đến mức nào

Ba bản trả lời khác nhau: hai bản lưu raw_input , một bản chỉ lưu hash. Nhóm chọn theo từng loại đầu vào, vì nhu cầu thực tế khác nhau:

- URL: lưu nguyên văn, vì người dùng cần mở lại lịch sử để xem mình đã kiểm tra link nào. Riêng log hệ thống chỉ ghi urlHash , tránh rò rỉ link có token trong query string.
- Text: lưu nguyên văn nhưng redact các pattern nhạy cảm (dãy 6 số giống OTP, số thẻ, mật khẩu) trước khi ghi DB.
- Phone và số tài khoản: chỉ lưu hash và bản masked. Không lưu số đầy đủ, kể cả trong bảng lịch sử.
- QR: lưu payload đã redact. Với VietQR thì mask số tài khoản.

2.4. Quy ước đặt key Redis

Dạng chung là {loại}:{miền}:{khóa} với loại thuộc cache , rate , lock , idem .

Key	TTL	Mục đích
cache:scan:url:{inputHash}	30 phút	Link scam thường lan theo nhóm chat, nhiều người gửi cùng một link. TTL ngắn vì site phishing bị takedown nhanh, kết quả cũ dễ sai
cache:scan:text:{inputHash}	5 phút	Chống retry khi mạng yếu
cache:entity:{type}:{valueHash}	15 phút	Tra cứu phone và số tài khoản
cache:entities:domain:{level}	10 phút	Whitelist và blacklist domain, worker tra mỗi lần scan

Key	TTL	Mục đích
cache:rules:active:{version}	5 phút	Bộ rule đang bật. Có {version} để ghi được ruleSnapshotVersion vào kết quả
cache:user:{userId}	5 phút	Profile người dùng
rate:{endpoint}:{scope}:{id}:{window}	theo endpoint	Đếm request
lock:scan:{inputType}:{inputHash}	60 giây	Nhiều request cùng một đầu vào chỉ sinh một job worker
idem:scan:{userId}:{idempotencyKey}	5 phút	Chống tạo trùng scan khi client bấm nhiều lần

Idempotency chốt TTL 5 phút. Bản 24 giờ bị bỏ vì kết quả scan hết hạn nhanh, giữ idempotency quá lâu sẽ trả lại kết quả cũ khi trang đã bị đánh sập.

Mọi key cache entity và rule phải bị invalidate ngay khi admin sửa dữ liệu tương ứng, không đợi TTL.

2.5. Hạn mức request

Bốn bản đưa ba bộ số khác nhau. Thống nhất một bảng:

Endpoint	Đã đăng nhập	Khách
POST /v1/auth/register , POST /v1/auth/login	5 request mỗi phút mỗi IP	như trên
POST /v1/scan/url	100 mỗi giờ mỗi user, 300 mỗi giờ mỗi IP	20 mỗi giờ mỗi IP
POST /v1/scan/text , /qr	200 mỗi giờ	50 mỗi giờ mỗi IP
POST /v1/scan/phone , /bank-account	200 mỗi giờ, thêm chặn 30 request trong 10 phút	20 mỗi giờ mỗi IP
POST /v1/reports	10 mỗi giờ	không cho phép

Riêng phone và bank-account có thêm giới hạn burst vì đây là hai endpoint dễ bị lợi dụng để dò danh sách số điện thoại hoặc số tài khoản của người khác. Vượt ngưỡng bất thường thì ghi audit_logs và cảnh báo admin.

3. Slice 1 - Auth, User và Audit log (Hùng)

Mục tiêu: định danh người dùng, phân quyền USER và ADMIN, ghi vết thao tác nhạy cảm. Đây là nền của các slice còn lại nên được làm trước.

Demo: đăng ký, đăng nhập nhận JWT, gọi API profile, để token hết hạn rồi refresh, sau đó thử vào route admin bằng tài khoản USER và nhận 403.

Frontend

- ☒ Trang đăng ký: form email, username, password, xác nhận mật khẩu. Validate ở client, lỗi từ backend hiển thị inline theo từng field.
- ☒ Trang đăng nhập: nhận username hoặc email. Lưu access token và refresh token sau khi đăng nhập thành công.
- ☒ Trang hồ sơ cá nhân: username, email, ngày tham gia, role, số lượt scan, nút đổi mật khẩu, danh sách scan gần nhất.
- ☒ Middleware Next.js chặn route `/admin/*`: không có token thì về `/login`, có token nhưng không phải ADMIN thì về trang chủ kèm thông báo. Đọc role từ JWT, không gọi thêm API.
- ☐ Màn hình đăng nhập trên mobile: lưu token vào Secure Storage, không dùng AsyncStorage thường. Đăng nhập bằng sinh trắc học để Phase 2.

API

- ☒ `POST /v1/auth/register` - validate email và độ dài mật khẩu, kiểm tra username trùng, hash BCrypt, trả 201 kèm thông tin cơ bản.
- ☒ `POST /v1/auth/login` - sinh access token TTL 15 phút và refresh token TTL 7 ngày, lưu hash refresh token vào DB. Sai 5 lần từ một IP thì chặn thêm 15 phút.
- ☒ `POST /v1/auth/refresh` - xác minh refresh token còn hạn, cấp access token mới và xoay vòng refresh token. Nếu một refresh token đã dùng bị dùng lại, thu hồi toàn bộ session của user đó.
- ☒ `POST /v1/auth/logout` - thu hồi refresh token của session hiện tại.
- ☒ `GET /v1/users/me` - trả profile theo JWT, không trả password hash hay refresh token.
- ☐ `PATCH /v1/users/me/password` - yêu cầu mật khẩu cũ, đổi xong thu hồi hết refresh token để buộc đăng nhập lại, ghi audit log.
- ☐ `GET /v1/admin/users` - danh sách user phân trang, filter theo role và trạng thái, chỉ ADMIN gọi được.

Worker

- ☐ Email Notification Worker, queue `notification.email` (Phase 2): gửi email xác thực sau khi đăng ký, dùng template tiếng Việt. Retry 3 lần rồi đẩy vào dead-letter queue.

Phần Auth xử lý đồng bộ là đủ, worker chỉ cần khi bổ sung xác thực email.

Dữ liệu

- ☒ Bảng `users`: `id` (UUID), `username`, `email`, `password_hash`, `role`, `status` (`ACTIVE`, `SUSPENDED`), `created_at`, `updated_at`. `username` và `email` unique.
- ☒ Bảng `refresh_tokens`: `user_id`, `token_hash`, `expires_at`, `revoked`. Lưu hash để nếu DB bị đọc trái phép thì token vẫn không dùng được.

- ☒ Bảng `audit_logs`: `user_id`, `action`, `ip_address`, `user_agent`, `metadata` (JSONB), `created_at`. Ghi cho đăng nhập, đăng xuất, đổi mật khẩu, xóa lịch sử scan, admin duyệt báo cáo, admin sửa rule hoặc entity.
- ☒ `rate:login:{ip}` TTL 15 phút và `rate:register:{ip}` TTL 1 phút.
- ☐ `cache:user:{userId}` TTL 5 phút.

Bảo mật

- ☒ Không lưu mật khẩu dạng plain text. BCrypt cost tối thiểu 10. Không ghi mật khẩu vào log kể cả dạng đã encode.
- ☒ Access token sống ngắn 15 phút để giảm thiệt hại nếu bị lộ. Refresh token sống 7 ngày nhưng lưu hash và có cơ chế thu hồi.
- ☒ JWT payload chỉ chứa `userId`, `role`, `iat`, `exp`. Không đưa email hay số điện thoại vào payload vì JWT chỉ được ký, không được mã hóa, ai có token đều đọc được.
- ☒ Rate limit trên hai endpoint auth để chống brute-force, trả 429 với thông báo dễ hiểu.
- ☒ Mọi route `/v1/admin/*` kiểm tra role ADMIN. USER nhận 403 với message ngắn, không tiết lộ cấu trúc hệ thống.
- ☒ `audit_logs` chỉ INSERT, không UPDATE và không DELETE. Admin cũng không được xóa log.
- ☐ Refresh token rotation với phát hiện reuse như mô tả ở endpoint refresh.

4. Slice 2 - Scan URL và SSRF guard (Khải)

Mục tiêu: chuẩn hóa link, lần theo redirect, kiểm tra TLS và tuổi tên miền, phân tích form trong HTML, rồi đưa đặc trưng sang Rule Engine chấm điểm. Toàn bộ chạy trong một lớp fetch an toàn, vì đây là module duy nhất của hệ thống chủ động gửi request ra Internet theo địa chỉ người dùng nhập.

Demo: dán `v1etcombank-verify.tk`, API trả `scanId` ngay, worker lần qua 2 redirect và phát hiện không có HTTPS hợp lệ, tên miền mới 5 ngày, gần giống `vietcombank.com.vn`, form yêu cầu OTP, trả 92 điểm mức DANGER kèm danh sách bằng chứng. Sau đó thử scan `http://169.254.169.254/latest/meta-data/` và hệ thống từ chối với lỗi `URL_TARGET_NOT_ALLOWED`.

Frontend

- ☐ Trang kiểm tra URL: ô dán link, validate sơ bộ ở client (chỉ http và https, tối đa 2048 ký tự). Sau khi gửi thì hiện trạng thái đang xử lý và tự poll theo `scanId`. Cho phép rời trang và xem lại trong lịch sử.
- ☐ Trang kết quả chi tiết URL: dùng lại component kết quả chung của Thắng, bổ sung phần riêng của URL gồm chuỗi redirect dạng timeline, thông tin chứng chỉ TLS, tuổi tên miền, danh sách field nhạy cảm trong form và tên miền thật mà nó đang nhái. Có nút gửi báo cáo và nút không đồng ý với kết quả.
- ☐ Trang lịch sử kiểm tra: bảng phân trang các lượt scan của chính người dùng, filter theo mức rủi ro và khoảng thời gian, có nút xóa lịch sử.

- ☐ Trang admin quản lý whitelist và blacklist tên miền: thêm, sửa, xóa, gán mức rủi ro, ghi nguồn, bật tắt. Có ô test nhanh để nhập một domain và xem sẽ khớp luật nào.
- ☐ Màn hình mobile scan URL, nhận link qua Share Target (phần Share Target do Thắng làm chung cho cả text và URL, xem mục 5).

API

- ☐ `POST /v1/scan/url` - nhận `{ url, idempotencyKey }`. Chuẩn hóa URL: lowercase host, xử lý punycode và IDN, decode percent-encoding, bỏ fragment, bỏ tham số tracking (`utm_*` , `fbclid`), sắp xếp query. Tính `inputHash = SHA-256(normalizedUrl)` rồi tra cache. Cache hit thì trả kết quả ngay với 200. Cache miss thì tạo `scan_requests` , publish vào queue `scan.url.requested` , trả 202 kèm `scanId` .
- ☐ `GET /v1/scan/{scanId}` - endpoint dùng chung cho cả bốn loại scan, Khỏi sở hữu vì gắn với `scan_requests` . Trả `PENDING` , `PROCESSING` , `COMPLETED` hoặc `FAILED` , kèm kết quả khi đã xong. Chỉ chủ sở hữu hoặc ADMIN xem được. Người không có quyền nhận 404 chứ không phải 403, để không tiết lộ là bản ghi có tồn tại.
- ☐ `GET /v1/scan/history` - phân trang theo cursor, filter theo `inputType` , mức rủi ro và khoảng ngày. Chỉ trả bản ghi của chính user.
- ☐ `DELETE /v1/scan/history` - xóa toàn bộ hoặc theo danh sách `scanId` , xóa cả file bằng chứng trên MinIO. Xóa thật, không soft-delete, vì đây là cam kết với người dùng. Ghi `audit_logs` với action `SCAN_HISTORY_DELETED` .
- ☐ `GET /v1/admin/entities` và `POST /v1/admin/entities` - quản lý whitelist và blacklist domain trong `risk_entities` . POST hỗ trợ thêm hàng loạt từ file. Sau khi ghi thì invalidate cache.

Worker

- ☐ URL Scanner Worker, queue `scan.url.requested` . Đây là phần nặng nhất của slice, nhận `{ scanId, normalizedUrl, inputHash }` rồi lần lượt:
 1. SSRF guard: resolve DNS và kiểm tra IP đích trước khi mở kết nối.
 2. Lần redirect tối đa 5 hop, ghi lại từng hop và kiểm tra lại IP sau mỗi hop để chống DNS rebinding.
 3. Kiểm tra TLS: có HTTPS hay không, CA nào, còn hạn không, hostname có khớp, có phải self-signed.
 4. Thông tin tên miền: tuổi domain qua WHOIS, TLD rủi ro như `.tk` , `.xyz` , `.top` , số lượng subdomain, dùng IP thay cho tên miền.
 5. So sánh typosquatting bằng Levenshtein và Jaro-Winkler với khoảng 100 thương hiệu phổ biến ở Việt Nam gồm ngân hàng, sàn thương mại điện tử, ví điện tử và cơ quan nhà nước.
 6. Parse HTML bằng Jsoup: đếm và phân loại input nhạy cảm (password, OTP, số thẻ, CCCD), form POST sang domain khác, iframe ẩn, form không dùng HTTPS.
 7. Gom đặc trưng thành `UrlFeatureSet` rồi gọi Rule Engine của Kiên để chấm điểm.

Ghi `risk_results`, cập nhật `scan_requests.status = COMPLETED`, set cache theo `inputHash`, lưu HTML snapshot lên MinIO. Retry 3 lần với backoff 2 giây, 8 giây, 30 giây cho lỗi mạng và timeout. Lỗi SSRF-blocked hoặc URL không hợp lệ thì fail ngay, không retry, vì retry cũng không đổi kết quả. Quá 3 lần thì vào `scan.url.dlq` và ghi `failure_reason`.

- ☐ Threat Feed Ingestion Worker, queue `threat.feed.sync` (Phase 2): scheduler chạy mỗi 6 giờ, nạp dữ liệu từ PhishTank, OpenPhish, URLhaus, Google Safe Browsing Lookup, chuẩn hóa rồi upsert vào `risk_entities`. Không ghi đè các entry do admin tự tạo. Một feed lỗi thì bỏ qua vòng đó, không làm hỏng dữ liệu đang có.

Dữ liệu

- ☐ Bảng `url_scan_details`: `scan_request_id`, `final_url`, `redirect_chain` (JSONB, mỗi hop kèm status code và IP), `http_status`, `tls_info` (JSONB), `domain_age_days`, `registrar`, `form_findings` (JSONB), `html_snapshot_key`, `fetch_duration_ms`. Tách khỏi `risk_results` để trang chi tiết truy vấn nhanh.
- ☐ Phần `entity_type = DOMAIN` và `URL` của bảng `risk_entities` dùng chung (chủ bảng là Thắng, xem mục 7).
- ☐ `cache:scan:url:{inputHash}`, `lock:scan:url:{inputHash}`, `cache:entities:domain:{level}` theo bảng ở mục 2.4.
- ☐ MinIO `evidence/url/{scanId}/page.html`: chỉ HTML tĩnh, không tải JS và tài nguyên ngoài, giới hạn 2 MB, tự xóa sau 30 ngày. Lưu để bảo vệ kết quả cảnh báo khi trang đã bị đánh sập hoặc chủ trang khiếu nại.
- ☐ MinIO `evidence/url/{scanId}/screenshot.png` (Phase 2): ảnh chụp trang cho admin xem khi kiểm duyệt mà không phải mở link thật. Phải render trong sandbox tách hoàn toàn khỏi mạng nội bộ.
- ☐ Bảng `feed_sync_logs` (Phase 2): số bản ghi thêm và cập nhật mỗi lần đồng bộ feed.

Bảo mật

- ☐ SSRF guard, phần quan trọng nhất của slice. Worker này gửi HTTP request tới địa chỉ do người dùng nhập, nếu chặn không đúng thì hệ thống trở thành công cụ để kẻ tấn công đọc mạng nội bộ. Quy tắc bắt buộc:
 - Chỉ scheme `http` và `https`. Chặn `file://`, `gopher://`, `ftp://`, `data:`.
 - Chỉ port 80 và 443.
 - Resolve DNS trước khi mở kết nối và chỉ kết nối tới đúng IP đã kiểm tra (IP pinning).
 - Chặn IP private và reserved: `10/8`, `172.16/12`, `192.168/16`, `127/8`, `0.0.0.0/8`, `169.254/16` (endpoint metadata của cloud), `::1`, `fc00::/7`, `fe80::/10`.
 - Kiểm tra lại IP sau mỗi redirect, tối đa 5 hop.
 - Timeout 5 giây mỗi hop, tổng tối đa 15 giây, response tối đa 2 MB.
 - Egress đi qua network riêng không có route tới subnet nội bộ.

- ☐ Không thực thi JavaScript, chỉ parse HTML tĩnh bằng Jsoup. Phase 1 không dùng headless browser để tránh rủi ro bị khai thác qua JS và tiết kiệm tài nguyên.
- ☐ Chỉ gửi một request GET tới trang đích. Không submit form, không tải file, không thao tác gì có thể gây tác động lên hệ thống của người khác. User-Agent ghi rõ mục đích và địa chỉ liên hệ, đây là chuẩn mực khi công cụ tự động truy cập web của người khác.
- ☐ Chỉ lưu cấu trúc form (có field tên `otp` , `password`), không lưu giá trị.
- ☐ Chặn scan cùng một domain quá nhiều lần trong thời gian ngắn, để hệ thống không bị dùng làm công cụ dò quét bên thứ ba.

5. Slice 3a - Rule Engine và Text Analyzer (Kiên)

Mục tiêu: phân tích tin nhắn, email, bài đăng mua bán hoặc tuyển dụng đáng nghi, và vận hành Rule Engine chấm điểm giải thích được cho toàn hệ thống. Cả URL Scanner và Entity Checker đều gọi vào Rule Engine, nên phần này là lõi dùng chung.

Demo: dán một tin nhắn giả mạo ngân hàng có kèm link và số điện thoại, yêu cầu cung cấp OTP. Hệ thống trích xuất entity, tra cứu số điện thoại và tên miền, Rule Engine cộng điểm và trả về mức DANGER kèm lý do từng rule cùng khuyến nghị không bấm link, không đọc OTP, không chuyển tiền.

Frontend

- ☐ Trang scan nội dung: textarea dán SMS, đoạn chat, email hoặc bài đăng. Cho chọn nguồn (`SMS` , `ZALO` , `MESSANGER` , `EMAIL` , `MARKETPLACE` , `OTHER`) vì cùng một nội dung đến từ SMS ngân hàng thì đáng ngờ hơn là từ tin nhắn bạn bè. Placeholder gợi ý ví dụ thật để người dùng biết nên dán gì.
- ☐ Trang kết quả phân tích nội dung: hiển thị điểm và mức rủi ro, các entity đã trích xuất, keyword và pattern khớp, kết quả cross-reference, và danh sách bằng chứng theo rule. Mỗi bằng chứng có một câu giải thích ngắn vì sao bị cộng điểm.
- ☐ Trang admin quản lý rule: danh sách rule đang bật, filter theo category, sửa trọng số, bật tắt, xem điều kiện.
- ☐ Màn hình mobile dán nội dung (nằm trong tab bar của Thắng, Kiên chỉ làm phần logic hiển thị kết quả text).
- ☐ Màn hình mobile kết quả tóm tắt: mức cảnh báo, 3 đến 5 lý do chính, việc nên làm tiếp, nút gửi báo cáo. Người dùng phổ thông không cần đọc hết danh sách bằng chứng.

API

- ☐ `POST /v1/scan/text` - nhận nội dung và `source` , validate tối đa 5000 ký tự, sanitize, tạo `scan_requests` . Nội dung ngắn thì xử lý đồng bộ, nội dung dài thì đẩy vào queue và trả `scanId` .
- ☐ `GET /v1/rules/active` - danh sách rule đang bật cho client hiển thị. Chỉ trả `code` , `name` , `category` , `weight` , không trả cấu trúc điều kiện, vì để lộ điều kiện chi tiết thì kẻ lừa đảo dễ viết tin nhắn né rule.

- ☐ GET `/v1/admin/rules` , POST `/v1/admin/rules` , PUT `/v1/admin/rules/{ruleId}` - CRUD rule, chỉ ADMIN. Mọi thay đổi ghi audit log và trigger nạp lại rule cache.
- ☐ POST `/v1/admin/rules/preview` - admin nhập payload mẫu và xem rule nào khớp, điểm cộng trừ và mức dự kiến, trước khi bật rule thật. Không ghi vào lịch sử scan của ai.

Worker

- ☐ Text Analyzer Worker, queue `scan.text.requested` . Các bước: tiền xử lý tiếng Việt ở mức heuristic, trích xuất URL, số điện thoại, số tài khoản, số tiền, pattern giống OTP và mốc thời gian gây áp lực; match keyword theo nhóm; match pattern tổ hợp như giả mạo ngân hàng, tuyển dụng giả, shipper giả, hoàn tiền giả; cross-reference URL và entity tìm được sang hai module kia; gọi Rule Engine tổng hợp. Retry 3 lần rồi vào `scan.text.dlq` và đánh dấu FAILED kèm lý do.
- ☐ Rule Cache Rebuild Worker, queue `rule.cache.rebuild` . Khi admin tạo, sửa hoặc bật tắt rule thì nạp lại bộ rule đang bật từ PostgreSQL và ghi vào Redis kèm version mới, để API và mọi worker dùng cùng một phiên bản. Rebuild lỗi thì giữ nguyên version cũ, không để hệ thống chạy với rule rỗng.

Dữ liệu

- ☐ Bảng `risk_rules` : `code` (unique), `name` , `category` , `weight` , `severity` , `enabled` , `description` , `explanation_template` , `recommendation_template` , `version` , `created_at` , `updated_at` . Hai cột template cho phép sinh câu giải thích và khuyến nghị bằng tiếng Việt tự nhiên mà không hardcode trong code.
- ☐ Bảng `rule_conditions` : `rule_id` , `condition_group` , `sequence` , `field_name` , `operator` (`CONTAINS` , `REGEX` , `EQUALS` , `GT` , `LT`), `value` , `logical_operator` . Hỗ trợ rule tổ hợp kiểu "text chứa keyword nhóm giả mạo ngân hàng, đồng thời có URL, đồng thời có yêu cầu OTP".
- ☐ Bảng `scan_requests` (Kiên sở hữu, xem mục 7).
- ☐ Bảng `risk_results` (Kiên sở hữu, xem mục 7), trong đó `analysis_json` cho text lưu `extractedEntities` , `keywordMatches` , `patternMatches` , `crossReferenceChecks` .
- ☐ `cache:scan:text:{inputHash}` và `cache:rules:active:{version}` .
- ☐ Từ điển keyword tiếng Việt theo nhóm: khẩn cấp và gây áp lực thời gian, giả danh cơ quan nhà nước, giả danh ngân hàng, yêu cầu thông tin nhạy cảm, yêu cầu chuyển tiền hoặc đặt cọc, hứa lợi ích bất thường. Nhóm cần tự thu thập từ tin nhắn thật, chưa có nguồn công khai nào cho tiếng Việt.

Bảo mật

- ☐ Redact pattern nhạy cảm trước khi lưu `analysis_json` và trước khi ghi log. Text Analyzer được phép phát hiện có OTP trong nội dung để cộng điểm, nhưng không lưu giá trị OTP.
- ☐ Không ghi `raw_input` vào application log. Log chỉ có `scanId` , `inputType` , độ dài và trạng thái. Nội dung tin nhắn người dùng dán vào có thể chứa OTP hoặc số thẻ của chính họ hoặc người khác.
- ☐ Mỗi điểm cộng hoặc trừ đều phải sinh một bằng chứng có `ruleCode` , tên rule, điểm và mô tả. Không có điểm nào không giải thích được. Đây cũng là lý do MVP chọn rule-based thay vì học máy: bảo vệ khóa luận cần giải thích được vì sao hệ thống chấm 92 điểm.

- ☐ Bộ rule bất biến trong một lần scan: job load rule một lần lúc bắt đầu và không nạp lại giữa chừng, đồng thời ghi `ruleSnapshotVersion` vào kết quả. Nếu admin sửa trọng số lúc đang xử lý mà job nạp lại thì hai nửa của cùng một kết quả sẽ dùng hai bộ rule khác nhau.
- ☐ Chỉ ADMIN được sửa rule. Ghi audit log mọi thay đổi trọng số và trạng thái rule, vì thay đổi này làm kết quả cảnh báo của cả hệ thống đổi theo.
- ☐ Giới hạn độ dài text và thời gian xử lý của worker.

6. Slice 3b - Entity Checker và QR Parser (Thắng)

Mục tiêu: tra cứu số điện thoại và số tài khoản ngân hàng rủi ro, giải mã QR và VietQR, và làm component hiển thị kết quả dùng chung cho cả bốn loại scan.

Demo: nhập một số tài khoản đã có báo cáo, hệ thống trả về số lần bị báo cáo, loại lừa đảo phổ biến, thời điểm gần nhất và khuyến nghị. Sau đó quét một QR VietQR chuyển tiền tới chính tài khoản đó và thấy cảnh báo xuất hiện trước khi người dùng bấm chuyển.

Frontend

- ☐ Trang kiểm tra số điện thoại: nhận cả `0xx` và `+84xx`, validate trước khi gọi API. Kết quả gồm điểm, số lượng báo cáo, loại lừa đảo phổ biến và thời điểm báo cáo gần nhất.
- ☐ Trang kiểm tra số tài khoản: nhập số tài khoản, chọn ngân hàng từ dropdown, tùy chọn nhập tên chủ tài khoản. Cảnh báo nổi bật nếu tài khoản nằm trong blacklist đã xác minh.
- ☐ Trang scan QR trên web: upload ảnh QR hoặc dán raw data. Hiển thị thông tin đã parse rồi chuyển sang kết quả tương ứng. Với VietQR thì hiện thêm ngân hàng, số tài khoản dạng masked và số tiền.
- ☐ Component hiển thị kết quả dùng chung: badge màu theo mức rủi ro, thanh điểm 0 đến 100, danh sách bằng chứng mở rộng thu gọn từng dòng, phần khuyến nghị và nút gửi báo cáo. Dùng cho URL, text, phone, bank và QR. Cần chốt props sớm vì ba người còn lại đều gọi vào.
- ☐ Trang tra cứu nhanh risk entity: tìm theo tên miền, số điện thoại hoặc số tài khoản, hiện trạng thái xác minh, số lần bị báo cáo và nguồn. Dành cho người muốn tra thủ công trước khi giao dịch.
- ☐ Màn hình mobile nhập thủ công: tab bar 4 tab gồm tin nhắn, số điện thoại, số tài khoản, QR. Kết quả hiện dạng bottom sheet.
- ☐ Màn hình quét QR bằng camera: xin quyền camera lúc dùng lần đầu, decode ngay trên máy bằng ZXing hoặc AVFoundation rồi chỉ gửi phần data đã decode lên server. Không upload ảnh.
- ☐ Share Target: đăng ký nhận text và URL từ Zalo, Messenger, SMS. Về kỹ thuật mỗi ứng dụng chỉ đăng ký được một Share Target ở cấp hệ điều hành, nên Thắng làm một cái duy nhất rồi route theo nội dung: là URL thì gọi `/scan/url` của Khải, còn lại gọi `/scan/text` của Kiên.

API

- ☐ POST `/v1/scan/phone` - chuẩn hóa về `+84xxxxxxxxx`, kiểm tra đầu số Việt Nam hợp lệ, tra `risk_entities` và `scam_reports`, tính điểm theo blacklist, số báo cáo, số báo cáo đã xác minh và độ mới

của báo cáo. Xử lý đồng bộ vì chỉ là lookup.

- ☐ `POST /v1/scan/bank-account` - chuẩn hóa số tài khoản (bỏ space và dấu gạch), validate `bankCode` theo danh sách ngân hàng Việt Nam, tra blacklist và lịch sử báo cáo.
- ☐ `POST /v1/scan/qr` - nhận `qrData` đã decode và `qrType` gợi ý (`AUTO` , `VIETQR` , `URL` , `TEXT`). Phân loại payload rồi route sang URL Scanner, Text Analyzer hoặc Bank Checker, cuối cùng gom thành một kết quả thống nhất. Giới hạn 2000 ký tự.
- ☐ `GET /v1/entities/search` - tìm risk entity theo keyword, filter theo loại và trạng thái, phân trang. Luôn mask giá trị trong response.

Worker

- ☐ Phone/Bank Checker Worker, queue `scan.entity.requested` . Xử lý các lookup cần tổng hợp nhiều báo cáo, tính điểm theo blacklist, số report, số report đã verify và hệ số giảm dần theo thời gian. Ghi bảng `entity_matches` , cập nhật phần cross-reference trong `analysis_json` , cache kết quả hay được tra. Nếu DB timeout thì để API trả trạng thái `PROCESSING` thay vì lỗi.
- ☐ QR Parser Worker, queue `scan.qr.requested` . Phân loại URL, VietQR, plain text hoặc không nhận dạng được. Với VietQR thì parse `bankCode` , `accountNumber` , `amount` , `content` rồi publish tiếp `scan.entity.requested` cho số tài khoản và `scan.text.requested` cho phần nội dung chuyển khoản. Sau khi các scan con xong thì gom lại. Payload không decode được thì kết thúc ở mức CAUTION với bằng chứng `QR_UNKNOWN_FORMAT` , không báo lỗi cho người dùng, vì QR lạ tự nó đã là dấu hiệu nên thận trọng.

Dữ liệu

- ☐ Bảng `risk_entities` (Thăng sở hữu): `entity_type` (`DOMAIN` , `URL` , `PHONE` , `BANK_ACCOUNT` , `KEYWORD` , `BRAND`), `value` , `value_hash` , `display_value_masked` , `risk_level` , `status` , `source` , `feed_name` , `report_count` , `verified_count` , `last_reported_at` , `notes` . Unique theo (`entity_type` , `value_hash`) , index cùng cặp đó để lookup nhanh.
- ☐ Bảng `entity_matches` : `scan_request_id` , `risk_entity_id` , `entity_type` , `matched_value_hash` , `matched_value_masked` , `match_source` , `score_contribution` . Cho phép thống kê sau này xem entity nào bị gặp lại nhiều nhất.
- ☐ `cache:entity:{type}:{valueHash}` TTL 15 phút, invalidate khi admin sửa entity hoặc khi một báo cáo được duyệt.
- ☐ Danh sách mã ngân hàng Việt Nam để validate `bankCode` , lấy theo chuẩn NAPAS.

Bảo mật

- ☐ Chuẩn hóa rồi hash số điện thoại và số tài khoản trước khi dùng làm khóa lookup, khóa cache hoặc ghi log. Không bao giờ dùng số nguyên văn làm cache key.
- ☐ Response public luôn mask: `+849***678` , `1234****7890` . Chỉ ADMIN xem được giá trị đầy đủ. Nếu không mask thì hệ thống thành công cụ để người này tra thông tin của người khác.
- ☐ Giới hạn kích thước đầu vào và reject ngay tại API: text 5000 ký tự, QR 2000 ký tự, số điện thoại và số tài khoản 20 ký tự.

- ☐ Chống dò quét: ngoài rate limit theo giờ còn có giới hạn burst 30 request trong 10 phút cho hai endpoint tra cứu. Ghi audit log khi một user hoặc IP tra trên 100 số trong 10 phút.
- ☐ Không khẳng định pháp lý. Khi một số điện thoại có báo cáo nhưng chưa được kiểm duyệt thì viết "đã có 3 báo cáo chưa được xác minh", không viết "số này là số lừa đảo". Đây là ranh giới quan trọng vì hệ thống có thể bị lợi dụng để hạ uy tín người khác.
- ☐ Không có tín hiệu rủi ro thì trả SAFE nhưng kèm câu nhắc rằng không có dữ liệu không có nghĩa là an toàn tuyệt đối.

7. Slice 4 - Community Report và Admin Dashboard (Hùng kiem, mức tối thiểu)

Mục tiêu: có nguồn dữ liệu do người dùng đóng góp để chấm điểm phone, số tài khoản và tên miền. Không có phần này thì `risk_entities` chỉ có dữ liệu admin tự nhập tay và ba slice kia không demo được đúng ý.

Demo: người dùng gửi báo cáo một số tài khoản kèm ảnh chụp tin nhắn, admin vào trang kiểm duyệt và duyệt, hệ thống tự tạo risk entity, sau đó người khác tra cùng số tài khoản đó và thấy cảnh báo.

Tính năng

- ☐ Form gửi báo cáo trên web và mobile: chọn loại đối tượng bị báo cáo, nhập giá trị, chọn loại lừa đảo, mô tả ngắn, đính kèm tối đa 3 ảnh.
- ☐ Nút gửi báo cáo trên mọi trang kết quả scan, điền sẵn đối tượng vừa scan.
- ☐ `POST /v1/reports` - tạo báo cáo, trạng thái ban đầu `PENDING`. Bắt buộc đăng nhập để có thể truy trách nhiệm nếu bị lạm dụng. Chặn báo cáo trùng cùng một đối tượng từ một user.
- ☐ `GET /v1/admin/reports` và `PUT /v1/admin/reports/{id}` - trang kiểm duyệt: xem chi tiết báo cáo và ảnh, duyệt hoặc từ chối kèm lý do. Khi duyệt thì upsert `risk_entities`, tăng `report_count` và `verified_count`, rồi invalidate cache entity.
- ☐ Trang dashboard admin: số lượt scan theo ngày, phân bố mức rủi ro, top entity bị báo cáo nhiều nhất, số báo cáo đang chờ duyệt. Chỉ cần bảng và biểu đồ đơn giản.
- ☐ Bảng `scam_reports`: `id`, `reporter_user_id`, `target_type`, `target_value`, `target_value_hash`, `scam_category`, `description`, `evidence_keys` (JSONB), `status` (`PENDING`, `VERIFIED`, `REJECTED`), `reviewed_by`, `reviewed_at`, `reject_reason`, `created_at`.
- ☐ MinIO `evidence/report/{reportId}/{n}.jpg`: ảnh do người dùng gửi, tối đa 5 MB mỗi ảnh, chỉ nhận JPEG và PNG.
- ☐ Chỉ ADMIN xem được ảnh bằng chứng của báo cáo, vì ảnh chụp tin nhắn thường chứa thông tin cá nhân của người thứ ba.
- ☐ Ghi audit log mọi lượt duyệt và từ chối, kèm `reviewed_by`. Một lượt duyệt sai có thể làm một số điện thoại hợp pháp bị gắn cờ rủi ro.
- ☐ Báo cáo chưa duyệt chỉ cộng điểm nhẹ, báo cáo đã duyệt mới cộng điểm nặng. Chống việc nhiều người cùng báo cáo sai một đối tượng để hạ uy tín.

Dời sang Phase 2: điểm uy tín người báo cáo, tự động duyệt khi đủ số lượng báo cáo từ user uy tín, export báo cáo định kỳ, thông báo cho người báo cáo khi báo cáo được duyệt.

8. Bảng dữ liệu dùng chung và chủ sở hữu

Vấn đề lớn nhất khi ghép bốn bản là ba người cùng khai ba bảng trung tâm với schema khác nhau. Nhóm chốt mỗi bảng có đúng một chủ, chủ bảng viết migration, ba người còn lại chỉ đọc và ghi qua repository của chủ bảng.

Bảng	Chủ	Ai dùng
users , refresh_tokens , audit_logs	Hùng	tất cả
scan_requests	Kiên	tất cả scanner
risk_results	Kiên	tất cả scanner
risk_rules , rule_conditions	Kiên	Rule Engine
risk_entities	Thắng	tất cả
entity_matches	Thắng	Slice 3a và 3b
url_scan_details	Khải	Slice 2
scam_reports	Hùng	Slice 2, 3a, 3b đọc để chấm điểm
feed_sync_logs	Khải (Phase 2)	Threat Feed Worker

Cột chính của hai bảng trung tâm:

- scan_requests : id (UUID), user_id (nullable vì cho phép scan không đăng nhập với hạn mức thấp), input_type , raw_input , normalized_input , input_hash , status , idempotency_key , failure_reason , created_at , completed_at . Index (user_id, created_at DESC) cho trang lịch sử và (input_hash, created_at DESC) cho tra cache.
- risk_results : id , scan_request_id (FK, quan hệ 1-1), risk_score , risk_level , confidence , summary , analysis_json , evidences_json , recommendation_json , rule_snapshot_version , created_at .

Ba cột JSONB được dùng vì mỗi loại đầu vào sinh ra cấu trúc phân tích khác nhau, và thêm rule mới thì không phải migration. Khi cần thống kê rule nào hay khớp nhất thì đánh GIN index lên evidences_json . Giữ rule_snapshot_version để sau này vẫn giải thích được vì sao một link từng bị chấm 92 điểm dù trọng số hiện tại đã đổi.

9. Luồng event giữa các worker

```
POST /v1/scan/url
  -> scan.url.requested          (URL Scanner, Khỏi)

POST /v1/scan/text
  -> scan.text.requested         (Text Analyzer, Kiên)
    - tìm thấy URL trong nội dung -> scan.url.requested
    - tìm thấy phone hoặc số TK   -> scan.entity.requested

POST /v1/scan/phone, /bank-account
  -> scan.entity.requested       (Entity Checker, Thắng)

POST /v1/scan/qr
  -> scan.qr.requested           (QR Parser, Thắng)
    - QR là URL      -> scan.url.requested
    - QR là VietQR   -> scan.entity.requested + scan.text.requested
    - QR là text     -> scan.text.requested

Admin sửa rule
  -> rule.cache.rebuild         (Rule Cache Rebuild, Kiên)
```

Hai interface phải chốt trước khi viết code, vì ba slice gọi qua lại nhau:

- `ScanJobPublisher.publishUrlScanRequested(...)` để Slice 3 gọi được URL Scanner mà không phụ thuộc vào cấu trúc bên trong của Slice 2.
- `RuleEngine.evaluate(FeatureSet) -> RiskResult` để URL Scanner và Entity Checker không tự viết logic chấm điểm riêng. Nếu mỗi người tự chấm thì ba loại scan sẽ cho ra thang điểm khác nhau.

Cách gom kết quả scan con: scan cha ở trạng thái `PROCESSING` và chờ tối đa 20 giây. Scan con nào chưa xong thì bỏ qua và ghi vào bảng chứng là chưa kiểm tra được, không để scan cha treo vô hạn.

10. Bộ luật rủi ro dự kiến

Trọng số dưới đây là bản đầu, sẽ hiệu chỉnh sau khi thử với dữ liệu thật. Điểm cộng dồn rồi chặn trong khoảng 0 đến 100.

Nhóm URL

Mã	Điều kiện	Trọng số
<code>URL_IN_BLACKLIST</code>	Khớp blacklist đã xác minh	+50

Mã	Điều kiện	Trọng số
DOMAIN_TYPOSQUATTING	Levenshtein tối đa 2 so với thương hiệu Việt Nam phổ biến	+30
URL_USES_IP	Dùng IP thay cho tên miền	+25
FORM_ASKS_SENSITIVE	Form có field OTP, mật khẩu, số thẻ, CCCD	+25
NO_HTTPS	Không có HTTPS hoặc lỗi chứng chỉ	+20
DOMAIN_TOO_NEW	Tuổi tên miền dưới 30 ngày	+20
SUSPICIOUS_CHARS	Có ký tự @ hoặc // bất thường trong URL	+15
TOO_MANY_SUBDOMAINS	Trên 3 subdomain	+15
MULTI_DOMAIN_REDIRECT	Redirect qua nhiều tên miền khác nhau	+10 mỗi hop
URL_TOO_LONG	Dài trên 75 ký tự	+10
DOMAIN_IN_WHITELIST	Khớp whitelist	-30

Nhóm nội dung

Mã	Điều kiện	Trọng số
TEXT_CONTAINS_BLACKLISTED_ENTITY	Có phone, số tài khoản hoặc tên miền trong blacklist đã xác minh	+40
TEXT_BANK_IMPERSONATION	Tổ hợp: nhắc tên ngân hàng, có link, yêu cầu xác thực hoặc cập nhật thông tin	+30
TEXT_ASKS_OTP	Yêu cầu cung cấp OTP hoặc mã xác thực	+30
TEXT_FAKE_AUTHORITY	Giả danh công an, thuế, điện lực, viện kiểm sát	+25
TEXT_ASKS_DEPOSIT	Yêu cầu chuyển cọc hoặc phí trước khi giao dịch	+20
TEXT_URGENCY_PRESSURE	Gây áp lực thời gian: khóa tài khoản trong 24 giờ, xử lý trong 2 giờ	+15
TEXT_TOO_GOOD_TO_BE_TRUE	Lương cao việc nhẹ, trúng thưởng, lãi cao không rủi ro	+15
TEXT_SHORTENED_LINK	Có link rút gọn	+10

Nhóm entity và QR

Mã	Điều kiện	Trọng số
ENTITY_IN_VERIFIED_BLACKLIST	Có trong blacklist đã kiểm duyệt	+50
ENTITY_MULTIPLE_REPORTS	+5 mỗi báo cáo đã duyệt	tối đa +30
ENTITY_RECENT_REPORT	Có báo cáo trong 7 ngày gần nhất	+10

Mã	Điều kiện	Trọng số
ENTITY_PENDING_REPORTS	Có báo cáo chưa duyệt	+5
PHONE_INVALID_VN_FORMAT	Đầu số không thuộc nhà mạng Việt Nam nào	+10
ENTITY_IN_WHITELIST	Trong whitelist	-40
QR_UNKNOWN_FORMAT	Không nhận dạng được định dạng QR	+10

Báo cáo cũ được nhân hệ số giảm dần: dưới 30 ngày tính đủ, 30 đến 180 ngày tính 75 phần trăm, trên 180 ngày tính 50 phần trăm. Lý do là một tài khoản bị báo cáo hai năm trước có thể đã đổi chủ.

11. Nguyên tắc bảo mật và quyền riêng tư toàn hệ thống

Bốn bản riêng đều nhắc những điểm dưới đây, nhóm xem đây là ràng buộc bắt buộc chứ không phải tùy chọn, và phải làm cùng lúc với tính năng chính.

- Chỉ xử lý dữ liệu người dùng chủ động gửi. Mobile không đọc SMS, thông báo, danh bạ, lịch sử gọi hay clipboard ở chế độ nền, và không dùng Accessibility Service. Nhóm chọn hướng này ngay từ đầu vì các quyền đó vừa khó xin trên store, vừa biến ứng dụng chống lừa đảo thành thứ đáng ngại hơn cả lừa đảo.
- Hash trước khi lưu, tra cứu hoặc ghi log với số điện thoại, số tài khoản và URL.
- Mask khi hiển thị cho người dùng thường. Chỉ ADMIN xem giá trị đầy đủ.
- Không lưu OTP, mật khẩu, CVV, số thẻ. Được phép phát hiện pattern để cộng điểm rồi redact trước khi lưu.
- Kết quả là đánh giá rủi ro, không phải kết luận pháp lý. Mọi response có disclaimer. Dùng cách nói "có dấu hiệu rủi ro", "đã có báo cáo chưa được xác minh", không dùng "kẻ lừa đảo" hay "chắc chắn là scam".
- Mọi cảnh báo phải truy được về rule cụ thể và bằng chứng cụ thể.
- Rate limit và idempotency trên mọi endpoint scan.
- Audit log mọi thay đổi rule, risk entity và mọi thao tác kiểm duyệt.
- Người dùng xóa được lịch sử scan của mình, xóa thật cả file bằng chứng liên quan.
- Có đường khiếu nại khi bị chấm sai: nút không đồng ý với kết quả trên trang kết quả, tạo bản ghi cho admin xem lại và có thể thêm vào whitelist. Không có cơ chế này thì một website hợp pháp bị chấm sai sẽ không có cách nào phản hồi.

12. Thứ tự triển khai

Slice 1 xong trước vì không phụ thuộc ai và cả ba slice còn lại cần JWT với `userId`. Phần này hiện đã chạy được.

Tiếp theo là ba việc phải làm song song và chốt xong trong tuần 5, vì mọi thứ sau đó đều chờ chúng: migration của `scan_requests` và `risk_results` (Kiên), migration `risk_entities` cùng component hiển thị kết quả (Thắng), và interface `RuleEngine.evaluate` (Kiên). Trước khi Rule Engine chạy được thì Khải vẫn làm được toàn bộ phân tích xuất đặc trưng URL, chỉ tạm chấm điểm bằng một implementation giả để test worker.

Sau đó Slice 2 và 3b làm song song, cùng gọi vào Rule Engine. Slice 4 làm sau cùng nhưng phải xong trước tuần demo, vì không có báo cáo đã duyệt thì phần chấm điểm phone và số tài khoản không có gì để chấm.

Tình trạng hiện tại:

Slice	Tình trạng
1 - Auth và User	Backend auth và phần web đã chạy. Còn đổi mật khẩu, trang admin users, mobile login
2 - Scan URL	Đã có danh sách tính năng, chưa viết code
3a - Rule Engine và Text	Đã có danh sách tính năng, chưa viết code
3b - Entity và QR	Đã có danh sách tính năng, chưa viết code
4 - Report và Dashboard	Mới định nghĩa phạm vi ở tài liệu này

13. Việc nhóm tự chốt trong tuần 5

- ☐ Viết file `Database_Schema.md` làm nguồn sự thật duy nhất cho DDL, thay vì để ba người tự khai bảng trong bốn tài liệu.
- ☐ Chốt props của component hiển thị kết quả để bốn loại scan dùng chung.
- ☐ Chốt hai interface ở mục 9 và viết implementation giả để bốn người test độc lập được.
- ☐ Thu thập dữ liệu mẫu tiếng Việt cho demo: link giả mạo ngân hàng, SMS giả mạo, số điện thoại đã bị báo cáo, số tài khoản bị báo cáo, QR chuyển khoản.
- ☐ Xây từ điển keyword tiếng Việt theo nhóm rủi ro. Việc này mất thời gian nhất vì không có nguồn công khai, phải đọc tin nhắn thật để rút ra.
- ☐ Lập danh sách khoảng 100 thương hiệu Việt Nam cho phần so sánh typosquatting, và quyết định ai duy trì.

14. Nội dung xin ý kiến thầy

- MVP có cần ứng dụng mobile thật hay web responsive kèm mô phỏng luồng mobile là đủ? Nhóm đang làm cả hai nhưng lo không đủ thời gian.
- Có nên tích hợp API bên ngoài thật (Google Safe Browsing, PhishTank, VirusTotal) hay dùng dữ liệu mẫu là đủ cho phạm vi khóa luận?

3. Nhóm chọn rule-based cho MVP thay vì học máy, để mọi cảnh báo đều giải thích được và không cần tập dữ liệu gán nhãn tiếng Việt. Hướng AI chỉ trình bày ở phần phát triển tiếp. Thầy thấy như vậy có đủ về mặt học thuật không?
4. Nhóm có 4 người nhưng phạm vi ban đầu chia 4 slice, và slice báo cáo cộng đồng lại là đầu vào của ba slice kia. Cách xử lý ở mục 1 là ghép slice đó vào slice Auth ở mức tối thiểu. Thầy xem cách chia này có hợp lý không.
5. Phần chạy ngầm trên mobile nên làm tới mức nào? Nhóm đã quyết định không đọc SMS và thông báo vì lý do quyền riêng tư, chỉ nhận dữ liệu khi người dùng chủ động chia sẻ. Việc này làm ứng dụng bớt tự động nhưng nhóm nghĩ là đánh đổi đúng.
6. Có nên làm thêm extension trình duyệt hoặc bot Zalo để tăng tính thực tế, hay nên tập trung hoàn thiện phần đang có?

Tổng hợp từ bốn tài liệu cá nhân tuần 4: `Hung_Feature.md` (01/08), `Khai_Feature.md` (09/08), `Kien_Feature.md` (03/08), `Thang_Feature.md` (01/08).